

docs_chain

Ни один отслеживаемый документ не может выйти из синхронизации — контент-хешированный, двунаправленный, атомарный.

Краткое описание

Docs Chain — универсальный движок распространения зависимостей между документами и базами данных, реализованный на Go. При изменении любого элемента зарегистрированной цепочки — исходника в Markdown, экспорта в формате HTML/PDF/DOCX или базы данных SQLite — система обнаруживает его по хешу содержимого и атомарно распространяет на все связанные элементы.

Короткое описание

Движок на базе Go, синхронизирующий документы и базы данных. Используя инкрементальные пересчёты с контент-хешированием в стиле Salsa на основе направленного ациклического графа (топологическая сортировка по алгоритму Кана, ранний отсев, двунаправленные рёбра синхронизации, атомарные переименования + транзакционные коммиты в SQLite), он пересоздаёт экспорты при изменении любого связанного артефакта.

Подробное описание

Docs Chain — это то, что создаётся, когда однажды надоедает раз за разом писать хрупкие скрипты на bash вроде «перегенерировать PDF при изменении Markdown». Он заменяет весь этот самопальный синхронизационный клей настоящим движком. Система моделирует документы и базы данных проекта как элементы цепочки и при изменении любого из них распространяет это изменение на все связанные элементы во всех объявленных направлениях, пересоздавая и переэкспортируя данные атомарно — так, чтобы ни один отслеживаемый артефакт никогда не выпадал

из синхронизации. Подход заимствует строгость у инкрементальных систем сборки, а не у скриптов: изменения обнаруживаются по **хешу содержимого, а не по времени модификации**, поэтому простая команда touch не запускает пересборку, а однобайтовое редактирование иницирует ровно те перестроения, которые должны произойти — без ложных срабатываний и упущенных изменений. Формально это можно выразить одной строкой: инкрементальные пересчёты с контент-хешированием в стиле Salsa на основе направленного ациклического графа с топологической сортировкой по алгоритму Кана, ранним отсеком неизменённых поддеревьев, объявленными двунаправленными рёбрами синхронизации с определением приоритета, а также атомарными переименованиями и транзакционными коммитами в SQLite, чтобы сбой в процессе распространения никогда не оставлял после себя наполовину записанный экспорт. Поставляется как submodule vasic-digital и используется как ключевая часть submodule HelixConstitution, поэтому любой проект, принимающий конституцию, получает Docs Chain «из коробки» и регистрирует собственные цепочки через YAML для каждого контекста. Реализация честно отражает статус (в соответствии с §11.4.6 конституции): Фазы 1–4 (ядро графа + хеширование, адаптеры/трансформации узлов, оркестратор распространения с атомарностью, конфигурируемый мультиконтекстный CLI с командами sync/verify/doctor/graph/watch) реализованы и протестированы; Фаза 4b добавляет встроенные двунаправленные преобразования md-to-sqlite/sqlite-to-md (на чистом Go, с детализацией до строк и байтово-стабильным круговым преобразованием), а также встроенную команду colorize-html; Фаза 5 — комплексное сквозное тестирование реальных бинарных данных — реализована и отмечена как УСПЕШНАЯ. Фазы 6–7 (распространение конституции, интеграция с ATMOSphere) остаются ЗАПЛАНИРОВАННЫМИ и доступны только при активации оператором. Первым реальным потребителем системы стал Herald, синхронизирующий корпус из 66 документов в нескольких форматах с подтверждённой чистотой синхронизации.

Зачем мы его создали

Документация, экспорты и базы данных расходятся в разные стороны, как только их начинают поддерживать вручную или с помощью ненадёжных скриптов. Docs Chain делает синхронизацию механической, точной по хешу содержимого и атомарной, так что изменение в любом месте цепочки корректно и безопасно обновляет всё, что находится ниже (и выше) по потоку.

Почему это меняет правила игры

Здесь воплощены те самые гарантии корректности, за которые годами борются авторы компиляторов и систем сборки — графы зависимостей с хешированием содержимого, минимальные пересчёты, атомарные коммиты — и всё это направлено на документацию и базы данных, область, которая до сих пор прозябала на крон-заданиях и благих намерениях. Настоящая двунаправленная синхронизация означает, что связь между исходником и его экспортом контролируется в обе стороны, так что фразы «документация устарела» и «экспорт не соответствует исходнику» перестают быть повторяющимися багами и становятся состояниями, которые движок просто не позволит создать.

Что нового

- Инкрементальный пересчёт по хешу содержимого (а не по времени модификации) на основе направленного ациклического графа с ранним обрывом цепочки.
- Двунаправленные синхронизирующие связи с объявленным источником истины (документация ↔ экспорты ↔ SQLite).
- Атомарные переименования + транзакционные коммиты через SQLite для безопасного распространения изменений даже при сбоях.
- Чистый Go-круговорот md-to-sqlite/sqlite-to-md с обнаружением расхождений на уровне строк.

Проблемы и решения

- **Ложные пересборки:** устранены за счёт определения изменений по хешу содержимого вместо временных меток.
- **Частичные/повреждённые обновления:** решены с помощью атомарных переименований и транзакций SQLite.
- **Корректный порядок обработки нескольких участников:** обеспечен топологической сортировкой Кана с ранним обрывом цепочки.
- **Честная отчётность о возможностях:** реализована маркировкой каждого этапа как РЕАЛИЗОВАН или ЗАПЛАНИРОВАН согласно §11.4.6.

Технологический стек (зачем и как)

- **Go** — весь движок (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + топологическая сортировка Кана** — упорядочивание зависимостей с ранним обрывом цепочки.

- **SQLite (чистый Go modernc)** — участники базы данных и транзакционные коммиты.
- **fsnotify** — демон watch для мгновенного распространения изменений.
- **Конфигурация YAML** — регистрация цепочек для каждого контекста.
- **exes: трансформеры** — подключаемые генераторы Markdown→HTML/PDF/DOCX.

Честность дорожной карты: Этапы 6-7 (распространение конституции, интеграция с ATMOSphere)
ЗАПЛАНИРОВАНЫ / доступны только оператору — пока не реализованы.